Week 6: Dependencies and backtracking

Prerequisites and possible choices

A pipeline stops when its jobs wait on each other. Find a feasible execution order, then enumerate the choices that can be made from a given set of items.

Use remaining prerequisites to schedule jobs. Choose, recurse, and restore to keep one search branch from changing the next.

Week 6 learning outcomes

Sessions 11 and 12: Indegree, cycles, choice paths, and restoration

- Check contracts and boundary conditions.
- Trace state and implement in Python.
- Explain correctness, time, auxiliary space, and output space.

Opening scenarios, traces, and checks are additional teaching examples.

Session 11 · Directed cycles and topological order

Order pipeline jobs while respecting prerequisites. Detect circular waiting and check disconnected components as well.

Prerequisites and Python semantics

A comprehension creates independent adjacency lists. Convert [after,before] to before→after. DFS states 0/1/2 mean unseen/active/completed.

Prerequisites and indegree

Input `[after,before]` represents edge `before→after`.

- `graph[before]`: neighbors released by completion
- `indegree[after]`: number of remaining prerequisites
- Indegree zero: a job that can run now
- Queue exhausted with fewer than V processed: cycle

Include independent jobs in the initial queue.

207 · Problem and state

LeetCode 207: Course Schedule · Medium

Decide whether all courses can be completed under prerequisite constraints.

`n=3, prerequisites=[[1,0],[2,1]] → True`

Direct edges before→after, remove zero-indegree nodes, and compare the removed count with V.

207 · Implementation

```
def can_finish(n, prerequisites):
    return len(course_order(n, prerequisites)) == n
```

207 · Correctness and cost

Each removed node has no remaining prerequisite. Every nonempty DAG has a zero-indegree node, so an unprocessed remainder must contain a cycle.

V is courses, E constraints. O(V+E) time and auxiliary space including the graph.

210 · Problem and state

LeetCode 210: Course Schedule II · Medium

Return any prerequisite-respecting order, or an empty list if impossible.

`n=4, prerequisites=[[2,0],[2,1],[3,2]] → [0,1,2,3]`

Record Kahn queue removals and require V nodes at the end.

210 · Implementation

```
def course_order(n, prerequisites):
    graph, indegree = [[] for _ in range(n)], [0] * n
    for after, before in prerequisites:
        graph[before].append(after)
        indegree[after] += 1
    queue = deque(v for v in range(n) if indegree[v] == 0)
    order = []
    while queue:
        node = queue.popleft()
        order.append(node)
        for neighbor in graph[node]:
            indegree[neighbor] -= 1
            if indegree[neighbor] == 0:
                queue.append(neighbor)
    return order if len(order) == n else []
```

210 · Correctness and cost

A course enters only after all prerequisites were removed, so every before precedes after in the result. A short result cannot be a complete order.

V is courses, E edges. O(V+E) time and total graph/queue/indegree/output space.

State trace

remove	indegree(0,1,2,3)	queue next
start	0,0,2,1	0,1
0	0,0,1,1	1
1	0,0,0,1	2
2	0,0,0,0	3
3	0,0,0,0	empty queue

A neighbor enters the queue exactly when its remaining indegree becomes zero.

Failure inputs and concept checks

[[1,0],[0,1]] is cyclic. A vertex in state 1 lies on the current search path. An edge back to it identifies a cycle. A vertex in state 2 has completed its search, so its result can be reused.

What fails if revisiting a completed node counts as a cycle?

The instructor answer is in the separate guide. Compare the reasoning line by line with the companion implementation.

Session 12 · Choose, recurse, restore

Generate combinations without counting reordered copies. Restore shared choices so one branch does not contaminate the next.

Prerequisites and Python semantics

Append, recurse, then pop the final choice. path.copy() stores a snapshot. pop() removes the most recent choice in O(1). remove(x) searches from the front and deletes the first equal value, taking O(n) in the worst case.

78 · Problem and state

LeetCode 78: Subsets · Medium

Return every subset of an array of distinct values.

`nums=[2,5] → [[],[2],[2,5],[5]]`

Choose an index at least start, recurse from the next index, pop, and store path.copy() at each call.

78 · Implementation

```
def subsets(nums):
    result, path = [], []

    def visit(start):
        result.append(path.copy())
        for index in range(start, len(nums)):
            path.append(nums[index])
            visit(index + 1)
            path.pop()

    visit(0)
    return result
```

78 · Correctness and cost

Every subset has one increasing index sequence. start permits exactly that sequence, and restoration returns the parent path to its prior state.

n is length. O(n·2^n) time/output including snapshots; O(n) path and call stack.

39 · Problem and state

LeetCode 39: Combination Sum · Medium

Return unique combinations reaching target with unlimited candidate reuse.

`candidates=[2,5], target=7 → [[2,5]]`

Sort candidates, recurse from the same index to permit reuse, and stop at values exceeding remaining.

39 · Implementation

```
def combinations(candidates, target):
    values, result, path = sorted(candidates), [], []

    def visit(start, remaining):
        if remaining == 0:
            result.append(path.copy())
            return
        for index in range(start, len(values)):
            value = values[index]
            if value > remaining:
                break
            path.append(value)
            visit(index, remaining - value)
            path.pop()

    visit(0, target)
    return result
```

39 · Correctness and cost

Positive choices decrease remaining and ensure termination. Nondecreasing indices avoid permutations while representing every combination.

m candidates,T target,c minimum,L=floor(T/c),B visited calls,S output elements. Time O(m log m+mB+S), auxiliary O(m+L), output O(S); conservatively B=O($(m+1)^{L+1}$).

State trace

action	path	start	saved result
visit(0)	[]	0	[]
choose 2	[2]	1	[2]
choose 5	[2,5]	2	[2,5]
pop 5, pop 2	[]	0	snapshot retained
choose 5	[5]	2	[5]

Saved paths are independent copies; popping restores the parent path.

Failure inputs and concept checks

result.append(path) stores one shared object; later pops can turn multiple results into empty lists.

Does returning from recursion terminate the parent for loop?

The instructor answer is in the separate guide. Compare the reasoning line by line with the companion implementation.

Two recursive index rules

Problem	Next start	Reason
Subsets	index+1	use each element at most once
Combination sum	index	permit candidate reuse

Neither problem moves backward in index, preventing reordered duplicates. Combination sum requires distinct positive candidates. Allowing zero can produce recursion that never decreases remaining.

Checkpoint questions

1. Find an execution order for `[[1,0],[0,1]]`.
2. Is `[1,0,2]` valid for `[[2,0],[2,1]]`?
3. How many subsets does `[2,5]` have, including the empty subset?
4. For candidates `[2]` and target 4, which index should the recursive call reuse?

Independent practice and connections

Student exercises

Hints and solutions

Instructor guide

Download Week 6 code

Submit code, state definition, correctness argument, complexity, and boundary inputs. Week 8 includes a mixed assessment without pattern labels.

Execution and sources

Run in the `code` folder of the weekly code archive.

```
python demo_11.py
python demo_12.py
python verify.py
python verify.py --student
```

Use the last command after implementing the student file.

Erik Demaine and Srini Devadas, MIT 6.006: Introduction to Algorithms (2011).